INDEX

Program No.	Programs	Page No.
1.	Write a C++ program to store details of 5 employees and display the details of the employee with the highest salary or search an employee by their ID.	1–3
2.	Program to demonstrate access control using protected members in public inheritance with the concept of class hierarchy (e.g., Vehicle → Car → Electric Car).	4–5
3.	Program to store and display information of 5 books using an array of objects.	6–8
4.	Program to demonstrate a function returning an object of a class Product that represents a combined discount of two products.	9–10
5.	Program to illustrate call by reference on objects where the function swaps the data of two Student objects.	11–12
6.	Program to sort names of 10 students in ascending order using an array.	13
7.	Program to perform linear search for a specific book title in an array of book names.	14
8.	Program to perform binary search on an array of roll numbers to find a student.	15
9.	Program to insert a new element at a given position in an integer array.	16
10.	Program to delete a specific number from an array and display the updated list.	17
11.	Write a C++ program to input a paragraph and a word, then find the first and last position of that word and count the number of digits present.	18
12.	Write a C++ program that finds the largest continuous sequence of even numbers between 1 and N.	19
13.	Write a C++ function to count how many times the words “computer” and “science” occur in a text file named "NOTES.TXT".	20–21
14.	Write a C++ program to display the total number of characters and lines in a file "DATA.TXT".	22
15.	Program to insert a new node at a specific position in a singly linked list.	23–24
16.	Program to delete the last node from a linked list and display the updated list.	25–26
17.	Write a C++ program to dynamically allocate memory for an integer array, accept values, and display their sum using pointers.	27
18.	Write a C++ program demonstrating runtime polymorphism by creating a base class Employee and derived classes Manager and Developer, each overriding a function calculateSalary().	28–29
19.	Write a C++ program with a class Car that uses a constructor to initialize its model and brand, and a destructor to display a message when an object is deleted.	30
20.	Write a C++ program to create a TXT file to store student records (Roll No., Name, Marks). Include options to update, delete, and view records using fstream.	31–34
21.	Write a menu-driven C++ program to implement queue operations (enqueue, dequeue, and display) using arrays and linked lists.	35–38
22.	Write a menu-driven C++ program to implement stack operations (push, pop, peek, and display) using a linear stack with arrays, a circular stack with arrays, and a stack with a linked list.	39–42

Program No.	Programs	Page No.
23.	Given the following table structure, write SQL commands to solve the questions below. Include the expected output for each query and explain the purpose of each SQL command used.	43–52

ID	Name	Role	Salary	Joining_Date	Company
1	Rahul Sharma	Developer	50000	2024-11-01	Skillarger
2	Anjali Mehta	Developer	45000	2024-10-15	Skillarger
3	Amitabh Sinha	Manager	70000	2024-09-20	Skillarger
4	Priya Singh	HR	40000	2024-07-10	Skillarger
5	Ravi Patel	Analyst	55000	2024-08-05	Skillarger

i) Write a SQL command to create the columns of above table and name it Employees.

ii) Write a SQL command to insert the necessary records to create above table.

iii) Write a SQL command to retrieve all the records.

iv) Write a SQL command to retrieve Name and Role columns from the table.

v) Write a SQL command to retrieve records of Salary greater than 50000.

vi) Write a SQL command to update Salary of record with ID 2 to 60000.

vii) Write a SQL command to delete record with ID 4.

viii) Write a SQL command to count total number of records of Employees.

ix) Write a SQL command to show records in descending order of Salary.

x) Write a SQL command to find the highest salary in Employees.

xi) Write a SQL command to retrieve number of Employees in each role.

xii) Write a SQL command to remove the table.

xiii) Write a SQL command to add a new column Email to Employees table.

xiv) Write a SQL command to update Email to an appropriate value.

PROGRAM 1 :

Write a C++ program to store details of 5 employees and display the details of the employee with the highest salary or search an employee by their ID.

SOLUTION :

```
#include <iostream>
#include <string>
using namespace std;

// ---------------- Employee Class ----------------
class Employee {
private:
    int id;
    string name;
    float salary;

public:
    // Input employee details
    void input() {
        cout << "Enter ID: ";
        cin >> id;
        cin.ignore();
        cout << "Enter Name: ";
        getline(cin, name);
        cout << "Enter Salary: ";
        cin >> salary;
    }

    // Display employee details
    void display() const {
        cout << "\nEmployee ID: " << id;
        cout << "\nName: " << name;
        cout << "\nSalary: $" << salary << endl;
    }

    // Getters
    int getID() const {
        return id;
    }

    float getSalary() const {
        return salary;
    }
};

// ---------------- EmployeeManager Class ----------------
class EmployeeManager {
private:
    Employee emp[5];

public:
    // Input details for all employees
    void inputEmployees() {
        cout << "Enter details for 5 employees:\n";
        for (int i = 0; i < 5; i++) {
            cout << "\nEmployee " << i + 1 << ":\n";
            emp[i].input();
        }
    }

    // Display employee with highest salary
    void displayHighestSalary() {
        int highestIndex = 0;
        for (int i = 1; i < 5; i++) {
            if (emp[i].getSalary() > emp[highestIndex].getSalary())
                highestIndex = i;
        }

        cout << "\nEmployee with highest salary:\n";
        emp[highestIndex].display();
    }
```

```
    // Search employee by ID
    void searchEmployee() {
        int searchID;
        bool found = false;

        cout << "\nEnter employee ID to search: ";
        cin >> searchID;

        for (int i = 0; i < 5; i++) {
            if (emp[i].getID() == searchID) {
                cout << "\nEmployee found:\n";
                emp[i].display();
                found = true;
                break;
            }
        }

        if (!found)
            cout << "\nEmployee with ID " << searchID << " not found.\n";
    }
};

// ---------------- Main Program ----------------
int main() {
    EmployeeManager manager;
    string again;

    manager.inputEmployees();

    do {
        int choice;
        cout << "\nChoose an option:\n";
        cout << "1. Display employee with the highest salary\n";
        cout << "2. Search employee by ID\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                manager.displayHighestSalary();
                break;

            case 2:
                manager.searchEmployee();
                break;

            default:
                cout << "\nInvalid choice.\n";
        }

        cout << "Do you want to try again? (Y/N): ";
        cin >> again;

    } while (again == "y" || again == "Y");

    return 0;
}
```

OUTPUT :

```
Enter details for 5 employees:

Employee 1:
Enter ID: 1
Enter Name: Deep
Enter Salary: 80000

Employee 2:
Enter ID: 2
Enter Name: Ram
Enter Salary: 20000

Employee 3:
Enter ID: 3
Enter Name: Rahul
Enter Salary: 40000

Employee 4:
Enter ID: 4
Enter Name: Karim
Enter Salary: 10000

Employee 5:
Enter ID: 5
Enter Name: Rahim
Enter Salary: 5000

Choose an option:
1. Display employee with the highest salary
2. Search employee by ID
Enter your choice: 1

Employee with highest salary:

Employee ID: 1
Name: Deep
Salary: $80000
Do you want to try again? (Y/N): Y

Choose an option:
1. Display employee with the highest salary
2. Search employee by ID
Enter your choice: 2

Enter employee ID to search: 3

Employee found:

Employee ID: 3
Name: Rahul
```

PROGRAM 2 :

Program to demonstrate access control using protected members in public inheritance with the concept of class hierarchy (e.g., Vehicle → Car → ElectricCar).

SOLUTION :

```
#include <iostream>
#include <string>
using namespace std;

// Base class: Vehicle
class Vehicle {
protected:
    string brand;   // Protected member: accessible in derived classes

public:
    void setBrand(string b) {
        brand = b;
    }

    void displayBrand() {
        cout << "Brand: " << brand << endl;
    }
};

// Derived class: Car (inherits publicly from Vehicle)
class Car : public Vehicle {
protected:
    int speed;   // Protected: accessible in derived classes, but not outside

public:
    void setCarDetails(string b, int s) {
        brand = b;   // Accessing protected member of Vehicle
        speed = s;
    }

    void showCarDetails() {
        cout << "Car Brand: " << brand << ", Speed: " << speed << " km/h" << endl;
    }
};

// Further derived class: ElectricCar (inherits publicly from Car)
class ElectricCar : public Car {
private:
    int batteryCapacity;   // in kWh

public:
    void setElectricCarDetails(string b, int s, int battery) {
        brand = b;   // Accessing protected member from Vehicle
        speed = s;   // Accessing protected member from Car
        batteryCapacity = battery;
    }

    void displayElectricCarDetails() {
        cout << "Electric Car Details:\n";
        cout << "Brand: " << brand << endl;
        cout << "Speed: " << speed << " km/h" << endl;
        cout << "Battery Capacity: " << batteryCapacity << " kWh" << endl;
    }
};

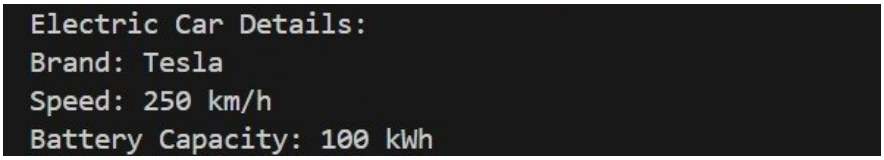
int main() {
    ElectricCar ecar;

    // Set and display details through public member functions
    ecar.setElectricCarDetails("Tesla", 250, 100);
    ecar.displayElectricCarDetails();

    // Demonstrate access restriction
    // ecar.brand = "Ford";  -> Error: 'brand' is protected and not accessible
    // ecar.speed = 200;     -> Error: 'speed' is protected and not accessible
```

```
    return 0;
}
```

OUTPUT :



PROGRAM 3 :

Program to store and display information of 5 books using an array of objects.

SOLUTION :

```
#include <iostream>
#include <string>
using namespace std;

// Class definition for Book
class Book {
private:
    int id;
    string title;
    string author;
    float price;

public:
    // Function to input book details
    void inputDetails() {
        cout << "Enter Book ID: ";
        cin >> id;
        cin.ignore();   // Clear newline from input buffer
        cout << "Enter Book Title: ";
        getline(cin, title);
        cout << "Enter Author Name: ";
        getline(cin, author);
        cout << "Enter Price: ";
        cin >> price;
        cin.ignore();   // Clear newline from input buffer after reading float
    }

    // Function to display book details
    void displayDetails() {
        cout << "\nBook ID: " << id;
        cout << "\nTitle: " << title;
        cout << "\nAuthor: " << author;
        cout << "\nPrice: $" << price << endl;
    }

    // Getter functions to retrieve book details
    int getId() {
        return id;
    }

    string getTitle() {
        return title;
    }

    string getAuthor() {
        return author;
    }

    float getPrice() {
        return price;
    }
};

int main() {
    Book books[5];   // Array of 5 Book objects

    cout << "Enter details of 5 books:\n";
    for (int i = 0; i < 5; i++) {
        cout << "\n--- Book " << i + 1 << " ---\n";
        books[i].inputDetails();
    }

    // Display individual book details
    cout << "\n========== Book Details ==========\n";
    for (int i = 0; i < 5; i++) {
        cout << "\n--- Book " << i + 1 << " ---";
        books[i].displayDetails();
```

```
    }

    // Display overall data summary
    float totalPrice = 0;
    cout << "\n========== Overall Data Summary ==========\n";
    for (int i = 0; i < 5; i++) {
        totalPrice += books[i].getPrice();
    }
    cout << "Total Price of all books: $" << totalPrice << endl;
    cout << "Average Price: $" << totalPrice / 5 << endl;

    // Additional Summary
    cout << "\nList of Books:\n";
    for (int i = 0; i < 5; i++) {
        cout << books[i].getTitle() << " by " << books[i].getAuthor() << endl;
    }

    return 0;
}
```

OUTPUT :

```
Enter details of 5 books:

--- Book 1 ---
Enter Book ID: 1
Enter Book Title: A
Enter Author Name: ABC
Enter Price: 1000

--- Book 2 ---
Enter Book ID: 2
Enter Book Title: B
Enter Author Name: BCD
Enter Price: 300

--- Book 3 ---
Enter Book ID: 3
Enter Book Title: C
Enter Author Name: CDE
Enter Price: 850

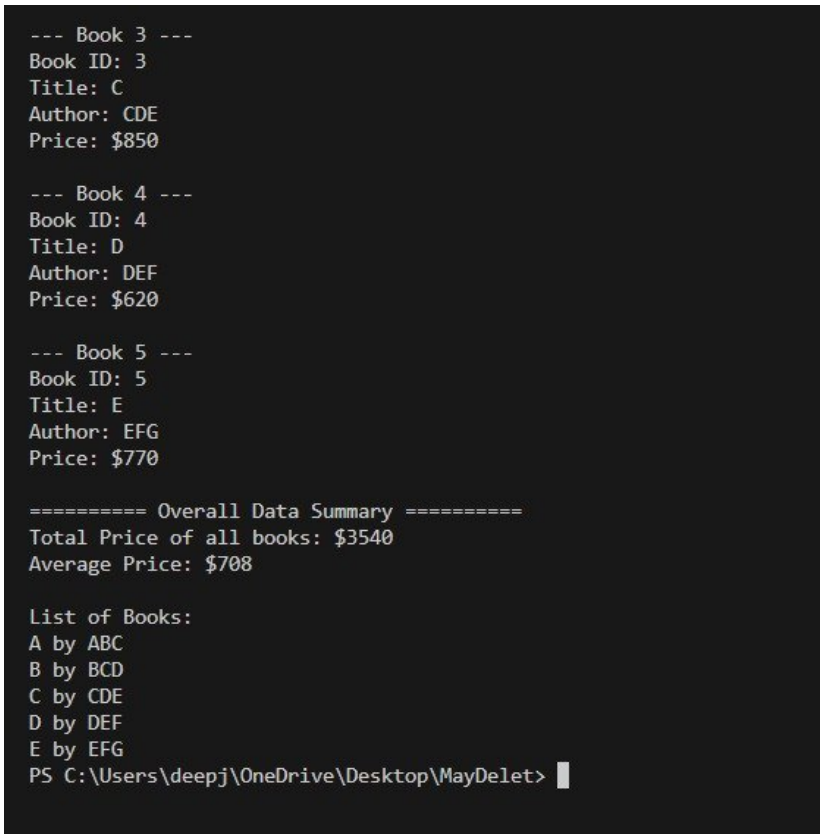
--- Book 4 ---
Enter Book ID: 4
Enter Book Title: D
Enter Author Name: DEF
Enter Price: 620

--- Book 5 ---
Enter Book ID: 5
Enter Book Title: E
Enter Author Name: EFG
Enter Price: 770

========== Book Details ==========

--- Book 1 ---
Book ID: 1
Title: A
Author: ABC
Price: $1000

--- Book 2 ---
Book ID: 2
Title: B
Author: BCD
Price: $300
```


--- Book 3 ---
Book ID: 3
Title: C
Author: CDE
Price: $850
--- Book 4 ---
Book ID: 4
Title: D
Author: DEF
Price: $620
--- Book 5 ---
Book ID: 5
Title: E
Author: EFG
Price: $770
========== Overall Data Summary ==========
Total Price of all books: $3540
Average Price: $708
List of Books:
A by ABC
B by BCD
C by CDE
D by DEF
E by EFG
PS C:\Users\deepj\OneDrive\Desktop\MayDelet>

PROGRAM 4 :

Program to demonstrate a function returning an object of a class Product that represents a combined discount of two products.

SOLUTION :

```
#include <iostream>
#include <string>
using namespace std;

class Product {
private:
    string name;
    float price;
    float discount;   // discount in percentage

public:
    // Default constructor
    Product() {
        name = "";
        price = 0;
        discount = 0;
    }

    // Parameterized constructor
    Product(string n, float p, float d) {
        name = n;
        price = p;
        discount = d;
    }

    // Function to input product details
    void input() {
        cout << "Enter product name: ";
        cin.ignore();
        getline(cin, name);
        cout << "Enter price: ";
        cin >> price;
        cout << "Enter discount (%): ";
        cin >> discount;
    }

    // Function to display product details
    void display() const {
        cout << "\nProduct Name: " << name;
        cout << "\nPrice: $" << price;
        cout << "\nDiscount: " << discount << "%";
        cout << "\nFinal Price: $" << price * (1 - discount / 100) << endl;
    }

    // Function to combine discounts of two products
    // Returns a new Product object
    Product combineDiscount(const Product &p2) const {
        Product temp;
        temp.name = "Combined Offer (" + name + " + " + p2.name + ")";

        // Average price and combined discount (for example purposes)
        temp.price = (price + p2.price) / 2;

        // Combined discount logic: sum but capped at 50%
        temp.discount = discount + p2.discount;
        if (temp.discount > 50)
            temp.discount = 50;   // limit maximum discount to 50%

        return temp;   // returning an object
    }
};

int main() {
    Product p1("Laptop", 800, 10);
    Product p2("Headphones", 200, 15);
```

```
    cout << "Product 1 details:";
    p1.display();

    cout << "\nProduct 2 details:";
    p2.display();

    // Function returning an object
    Product combined = p1.combineDiscount(p2);

    cout << "\nCombined Product details:";
    combined.display();

    return 0;
}
```

OUTPUT :

```
Product 1 details:
Product Name: Laptop
Price: $800
Discount: 10%
Final Price: $720

Product 2 details:
Product Name: Headphones
Price: $200
Discount: 15%
Final Price: $170

Combined Product details:
Product Name: Combined Offer (Laptop + Headphones)
Price: $500
Discount: 25%
Final Price: $375
```

PROGRAM 5 :

Program to illustrate call by reference on objects where the function swaps the data of two Student objects.

SOLUTION :

```
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    int rollNo;
    string name;

public:
    // Function to input student details
    void input() {
        cout << "Enter Roll Number: ";
        cin >> rollNo;
        cin.ignore();   // clear newline from input buffer
        cout << "Enter Name: ";
        getline(cin, name);
    }

    // Function to display student details
    void display() const {
        cout << "Roll No: " << rollNo << ", Name: " << name << endl;
    }

    // Friend function to swap two Student objects (Call by Reference)
    friend void swapData(Student &s1, Student &s2);
};

// Function to swap data of two students using call by reference
void swapData(Student &s1, Student &s2) {
    // Swap data members using a temporary object
    Student temp = s1;
    s1 = s2;
    s2 = temp;
}

int main() {
    Student s1, s2;

    cout << "Enter details for Student 1:\n";
    s1.input();

    cout << "\nEnter details for Student 2:\n";
    s2.input();

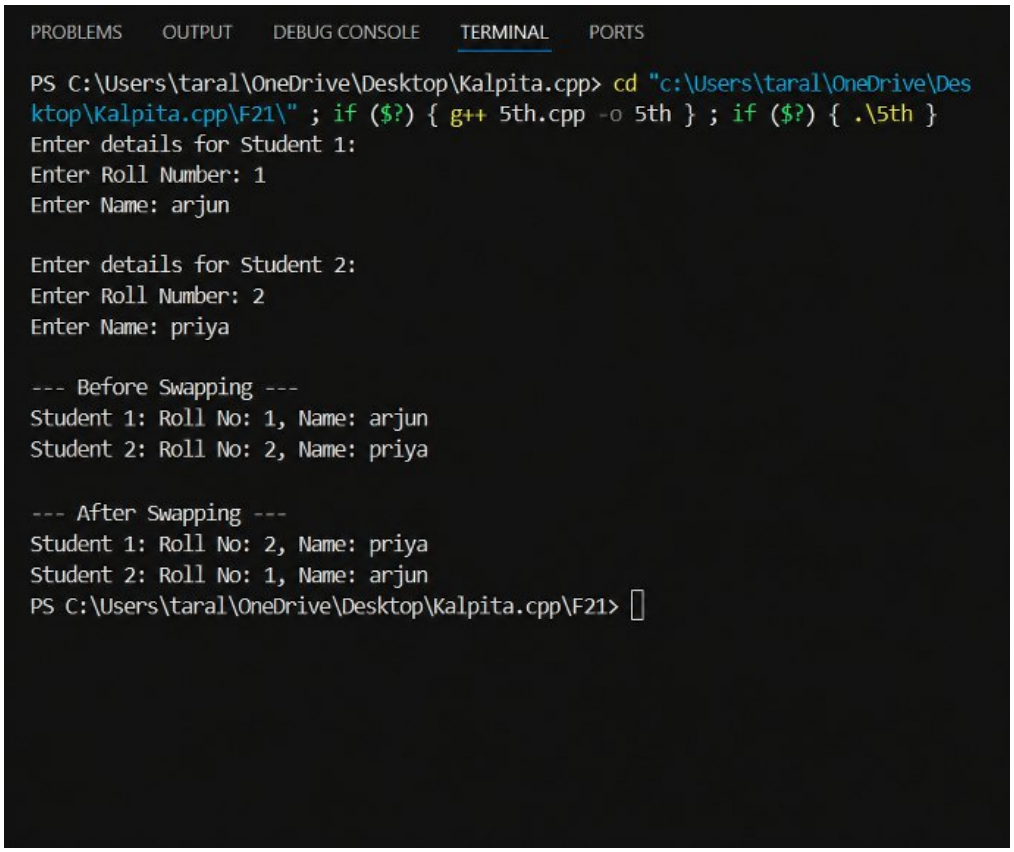
    cout << "\n--- Before Swapping ---\n";
    cout << "Student 1: ";
    s1.display();
    cout << "Student 2: ";
    s2.display();

    // Function call by reference (objects are swapped)
    swapData(s1, s2);

    cout << "\n--- After Swapping ---\n";
    cout << "Student 1: ";
    s1.display();
    cout << "Student 2: ";
    s2.display();

    return 0;
}
```

OUTPUT :


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\taral\OneDrive\Desktop\Kalpita.cpp> cd "c:\Users\taral\OneDrive\Desktop\Kalpita.cpp\F21\" ; if ($?) { g++ 5th.cpp -o 5th } ; if ($?) { .\5th }
Enter details for Student 1:
Enter Roll Number: 1
Enter Name: arjun
Enter details for Student 2:
Enter Roll Number: 2
Enter Name: priya
--- Before Swapping ---
Student 1: Roll No: 1, Name: arjun
Student 2: Roll No: 2, Name: priya
--- After Swapping ---
Student 1: Roll No: 2, Name: priya
Student 2: Roll No: 1, Name: arjun
PS C:\Users\taral\OneDrive\Desktop\Kalpita.cpp\F21>

PROGRAM 6 :

Program to sort names of 10 students in ascending order using an array.

SOLUTION :

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string names[10];

    cout << "Enter names of 10 students:\n";
    for (int i = 0; i < 10; i++) {
        cout << "Student " << i + 1 << ": ";
        getline(cin, names[i]);
    }

    // Sorting names in ascending order (alphabetical order)
    for (int i = 0; i < 9; i++) {
        for (int j = i + 1; j < 10; j++) {
            if (names[i] > names[j]) {
                // swap names
                string temp = names[i];
                names[i] = names[j];
                names[j] = temp;
            }
        }
    }

    cout << "\nNames of students in ascending order:\n";
    for (int i = 0; i < 10; i++) {
        cout << names[i] << endl;
    }

    return 0;
}
```

OUTPUT :

```
Enter names of 10 students:
Student 1: AB
Student 2: BC
Student 3: CD
Student 4: DE
Student 5: EF
Student 6: FG
Student 7: GH
Student 8: HI
Student 9: IJ
Student 10: JK

Names of students in ascending order:
AB
BC
CD
DE
EF
FG
GH
HI
IJ
JK
```

PROGRAM 7 :

Program to perform linear search for a specific book title in an array of book names.

SOLUTION :

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    const int SIZE = 5;   // Number of books (you can change this)
    string books[SIZE];
    string searchTitle;
    bool found = false;

    // Input book titles
    cout << "Enter titles of " << SIZE << " books:\n";
    for (int i = 0; i < SIZE; i++) {
        cout << "Book " << i + 1 << ": ";
        getline(cin, books[i]);
    }

    // Input title to search
    cout << "\nEnter the book title to search: ";
    getline(cin, searchTitle);

    // Linear search
    for (int i = 0; i < SIZE; i++) {
        if (books[i] == searchTitle) {
            cout << "\nBook found at position " << i + 1 << "." << endl;
            found = true;
            break;
        }
    }

    if (!found)
        cout << "\nBook not found in the list." << endl;

    return 0;
}
```

OUTPUT :

```
Enter titles of 5 books:
Book 1: AAA
Book 2: BBB
Book 3: CCC
Book 4: DDD
Book 5: EEE

Enter the book title to search: DDD

Book found at position 4.
```

PROGRAM 8 :

Program to perform binary search on an array of roll numbers to find a student.

SOLUTION :

```
#include <iostream>
using namespace std;

int main() {
    const int SIZE = 10;
    int rollNumbers[SIZE];
    int key;
    int low, high, mid;
    bool found = false;

    // Input roll numbers (must be sorted)
    cout << "Enter " << SIZE << " roll numbers in ascending order:\n";
    for (int i = 0; i < SIZE; i++) {
        cout << "Roll number " << i + 1 << ": ";
        cin >> rollNumbers[i];
    }

    // Input roll number to search
    cout << "\nEnter roll number to search: ";
    cin >> key;

    // Initialize binary search bounds
    low = 0;
    high = SIZE - 1;

    // Binary Search Algorithm
    while (low <= high) {
        mid = (low + high) / 2;

        if (rollNumbers[mid] == key) {
            cout << "\nStudent found at position " << mid + 1 << " in the list." << endl;
            found = true;
            break;
        }
        else if (rollNumbers[mid] < key)
            low = mid + 1;    // Search right half
        else
            high = mid - 1;   // Search left half
    }

    if (!found)
        cout << "\nStudent with roll number " << key << " not found." << endl;

    return 0;
}
```

OUTPUT :

```
Enter 10 roll numbers in ascending order:
Roll number 1: 1
Roll number 2: 2
Roll number 3: 3
Roll number 4: 4
Roll number 5: 5
Roll number 6: 6
Roll number 7: 7
Roll number 8: 8
Roll number 9: 9
Roll number 10: 10

Enter roll number to search: 6

Student found at position 6 in the list.
```

PROGRAM 9 :

Program to insert a new element at a given position in an integer array.

SOLUTION :

```
#include <iostream>
using namespace std;

int main() {
    int arr[20];   // array with extra space for insertion
    int n, pos, num;

    // Input array size
    cout << "Enter number of elements (max 19): ";
    cin >> n;

    // Input array elements
    cout << "Enter " << n << " elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // Input new element and position
    cout << "\nEnter the element to insert: ";
    cin >> num;

    cout << "Enter the position (1 to " << n + 1 << "): ";
    cin >> pos;

    // Validate position
    if (pos < 1 || pos > n + 1) {
        cout << "Invalid position!" << endl;
        return 0;
    }

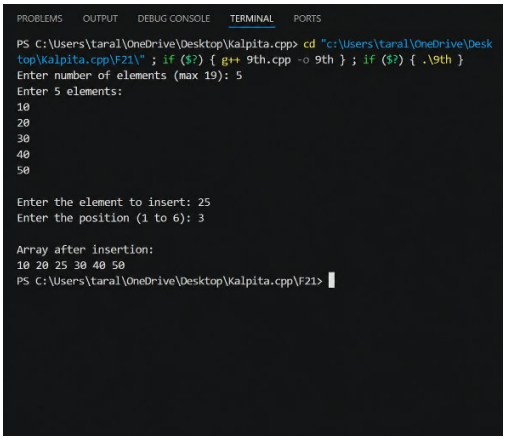
    // Shift elements to the right to make space
    for (int i = n; i >= pos; i--) {
        arr[i] = arr[i - 1];
    }

    // Insert the new element
    arr[pos - 1] = num;
    n++;   // increase array size

    // Display updated array
    cout << "\nArray after insertion:\n";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

OUTPUT :



PROGRAM 10 :

Program to delete a specific number from an array and display the updated list.

SOLUTION :

```
#include <iostream>
using namespace std;

int main() {
    int arr[20];
    int n, num, pos = -1;

    // Input array size
    cout << "Enter number of elements (max 20): ";
    cin >> n;

    // Input array elements
    cout << "Enter " << n << " elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // Input number to delete
    cout << "\nEnter the number to delete: ";
    cin >> num;

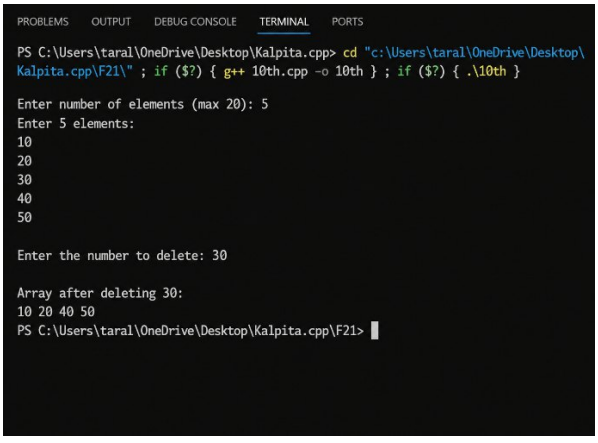
    // Find position of the number to delete
    for (int i = 0; i < n; i++) {
        if (arr[i] == num) {
            pos = i;
            break;
        }
    }

    if (pos == -1) {
        cout << "\nNumber not found in the array.\n";
    } else {
        // Shift elements to the left to remove the element
        for (int i = pos; i < n - 1; i++) {
            arr[i] = arr[i + 1];
        }
        n--;   // Decrease size after deletion

        // Display updated array
        cout << "\nArray after deleting " << num << ":\n";
        for (int i = 0; i < n; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

OUTPUT :



PROGRAM 11 :

Write a C++ program to input a paragraph and a word, then find the first and last position of that word and count the number of digits present.

SOLUTION :

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string paragraph, word;
    int firstPos = -1, lastPos = -1, digitCount = 0;

    // Input paragraph
    cout << "Enter a paragraph:\n";
    getline(cin, paragraph);

    // Input word to search
    cout << "Enter the word to search: ";
    getline(cin, word);   // use getline instead of cin >>

    // Search for first and last positions
    size_t pos = paragraph.find(word);
    if (pos != string::npos)
        firstPos = pos;

    pos = paragraph.rfind(word);
    if (pos != string::npos)
        lastPos = pos;

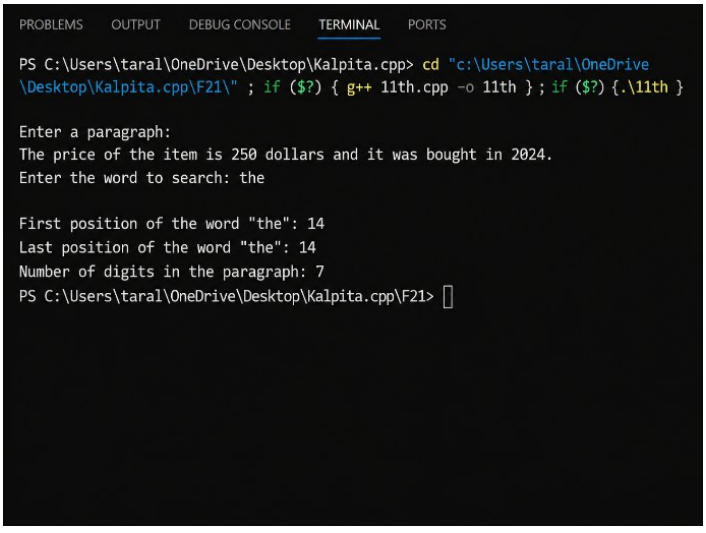
    // Count digits in paragraph
    for (size_t i = 0; i < paragraph.length(); i++) {
        if (isdigit(paragraph[i]))
            digitCount++;
    }

    // Display results
    if (firstPos != -1) {
        cout << "\nFirst position of the word \"" << word << "\": " << firstPos + 1 << endl;
        cout << "Last position of the word \"" << word << "\": " << lastPos + 1 << endl;
    } else {
        cout << "\nWord \"" << word << "\" not found in the paragraph." << endl;
    }

    cout << "Number of digits in the paragraph: " << digitCount << endl;

    return 0;
}
```

OUTPUT :



PROGRAM 12 :

Write a C++ program that finds the largest continuous sequence of even numbers between 1 and N.

SOLUTION :

```
#include <iostream>
using namespace std;

int main() {
    int N;
    cout << "Enter a number: ";
    cin >> N;

    const int firstEven = 2;
    int lastEven = (N % 2 == 0) ? N : N - 1;

    if (lastEven < 2) {
        cout << "No even numbers found between 1 and " << N << endl;
        return 0;
    }

    cout << "Largest continuous sequence of even numbers:\n";

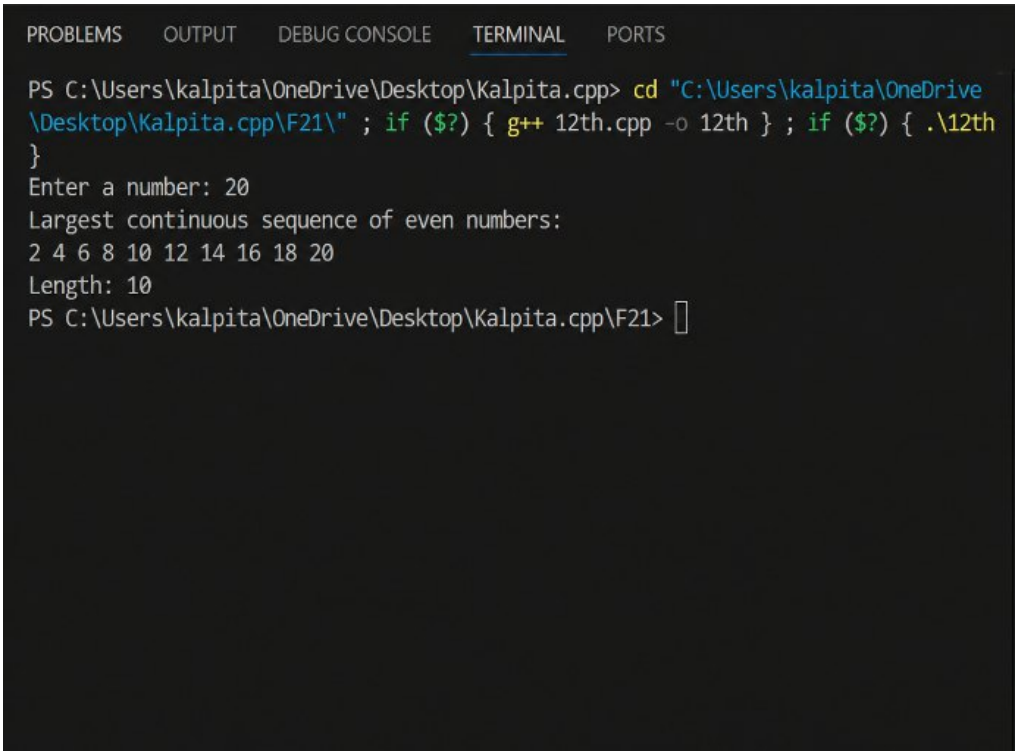
    for (int i = firstEven; i <= lastEven; i += 2) {
        cout << i << " ";
    }

    int length = lastEven / 2;

    cout << "\nLength: " << length << endl;

    return 0;
}
```

OUTPUT :



PROGRAM 13 :

Write a C++ function to count how many times the words “computer” and “science” occur in a text file named "NOTES.TXT".

SOLUTION :

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

// Convert a string to lowercase manually
string toLower(string s) {
    for (size_t i = 0; i < s.length(); i++) {
        if (s[i] >= 'A' && s[i] <= 'Z') {
            s[i] = s[i] + ('a' - 'A');   // convert uppercase to lowercase
        }
    }
    return s;
}

// Count occurrences of "computer" and "science" in NOTES.TXT
void countWordsInNotes() {
    ifstream fin("NOTES.TXT");
    if (!fin) {
        cout << "Error: Could not open NOTES.TXT\n";
        return;
    }

    int computerCount = 0;
    int scienceCount = 0;
    string word;

    while (fin >> word) {
        string lower = toLower(word);

        // Remove punctuation at start
        while (!lower.empty() &&
               (lower.front() < 'a' || lower.front() > 'z')) {
            lower.erase(0, 1);
        }

        // Remove punctuation at end
        while (!lower.empty() &&
               (lower.back() < 'a' || lower.back() > 'z')) {
            lower.pop_back();
        }

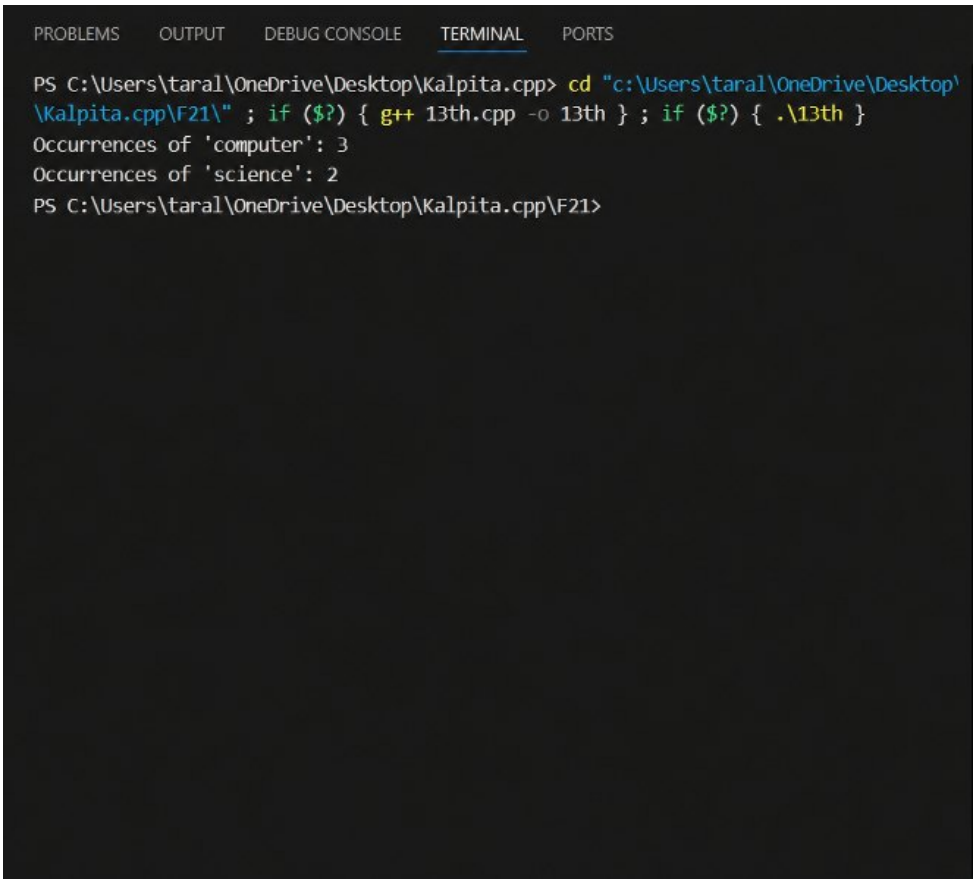
        if (lower == "computer") computerCount++;
        else if (lower == "science") scienceCount++;
    }

    fin.close();

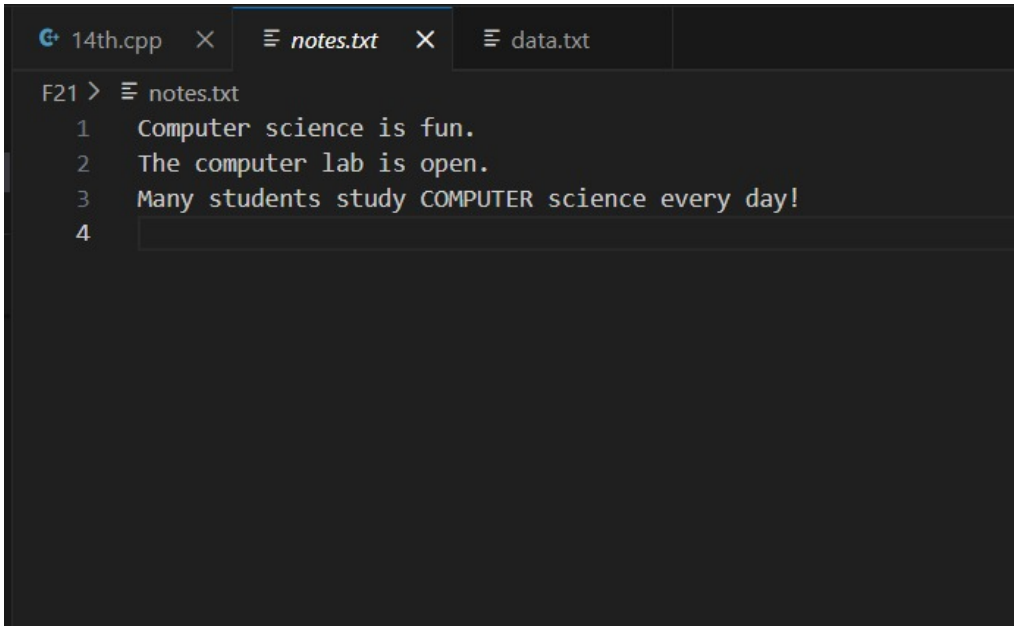
    cout << "Occurrences of 'computer': " << computerCount << endl;
    cout << "Occurrences of 'science': " << scienceCount << endl;
}

int main() {
    countWordsInNotes();
    return 0;
}
```

OUTPUT :



NOTES.TXT :



PROGRAM 14 :

Write a C++ program to display the total number of characters and lines in a file "DATA.TXT".

SOLUTION :

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream file("DATA.TXT");
    if (!file) {
        cout << "Error: Cannot open file DATA.TXT\n";
        return 1;
    }

    string line;
    int lineCount = 0;
    int charCount = 0;

    while (getline(file, line)) {
        lineCount++;
        charCount += line.length();   // characters in the line (excluding newline)
        charCount++;                  // count the newline character
    }

    file.close();

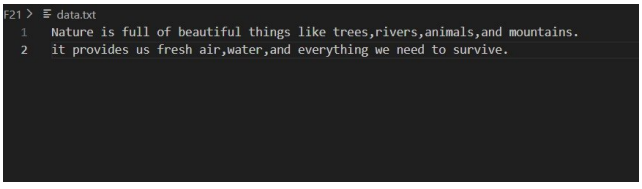
    cout << "Total lines: " << lineCount << endl;
    cout << "Total characters: " << charCount << endl;

    return 0;
}
```

OUTPUT :



DATA.TXT :



PROGRAM 15 :

Program to insert a new node at a specific position in a singly linked list.

SOLUTION :

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

void insertAtPosition(Node*& head, int data, int position) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->next = nullptr;

    if (position == 1) {
        newNode->next = head;
        head = newNode;
        return;
    }

    Node* temp = head;
    for (int i = 1; i < position - 1 && temp != nullptr; i++) {
        temp = temp->next;
    }

    if (temp == nullptr) {
        cout << "Position out of range!" << endl;
        delete newNode;
        return;
    }

    newNode->next = temp->next;
    temp->next = newNode;
}

void printList(Node* head) {
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

int main() {
    Node* head = nullptr;

    head = new Node{10, nullptr};
    head->next = new Node{20, nullptr};
    head->next->next = new Node{30, nullptr};

    cout << "Original List: ";
    printList(head);

    int data, position;
    cout << "\nEnter data to insert: ";
    cin >> data;
    cout << "Enter position to insert at: ";
    cin >> position;

    insertAtPosition(head, data, position);

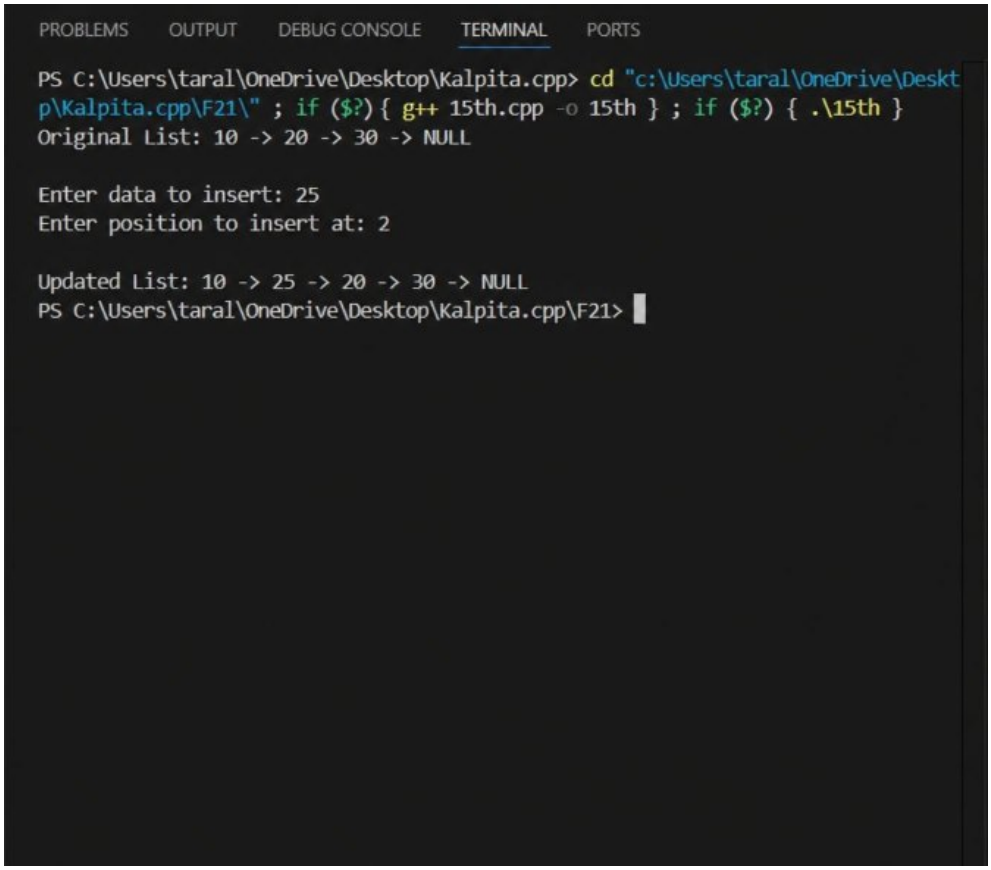
    cout << "\nUpdated List: ";
    printList(head);

    Node* temp;
    while (head != nullptr) {
        temp = head;
```

```
        head = head->next;
        delete temp;
    }

    return 0;
}
```

OUTPUT :



PROGRAM 16 :

Program to delete the last node from a linked list and display the updated list.

SOLUTION :

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

void deleteLastNode(Node*& head) {
    if (head == nullptr) {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    if (head->next == nullptr) {
        delete head;
        head = nullptr;
        cout << "Last node deleted. List is now empty." << endl;
        return;
    }

    Node* temp = head;
    while (temp->next->next != nullptr) {
        temp = temp->next;
    }

    delete temp->next;
    temp->next = nullptr;
    cout << "Last node deleted successfully." << endl;
}

void printList(Node* head) {
    if (head == nullptr) {
        cout << "List is empty." << endl;
        return;
    }

    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

int main() {
    Node* head = nullptr;
    head = new Node{10, nullptr};
    head->next = new Node{20, nullptr};
    head->next->next = new Node{30, nullptr};
    head->next->next->next = new Node{40, nullptr};

    cout << "Original List: ";
    printList(head);
    deleteLastNode(head);

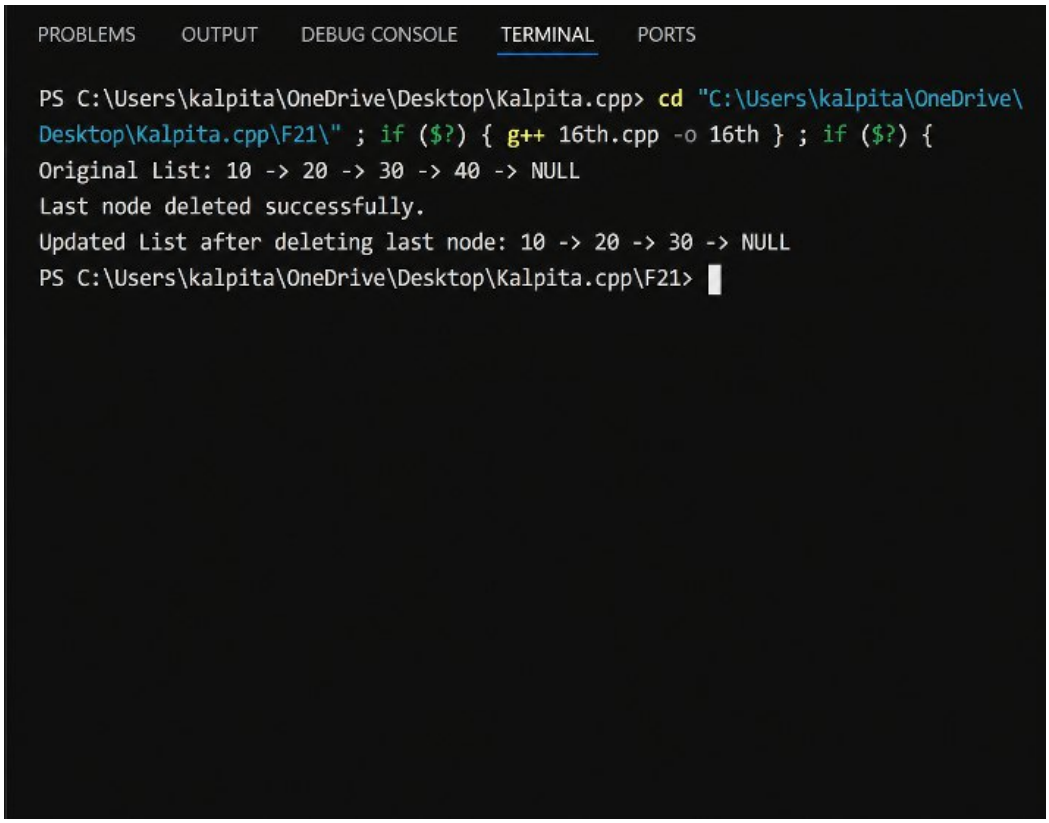
    cout << "Updated List after deleting last node: ";
    printList(head);

    Node* temp;
    while (head != nullptr) {
        temp = head;
        head = head->next;
        delete temp;
    }

    return 0;
```

```
}
```

OUTPUT :


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\kalpita\OneDrive\Desktop\Kalpita.cpp> cd "C:\Users\kalpita\OneDrive\
Desktop\Kalpita.cpp\F21\" ; if ($?) { g++ 16th.cpp -o 16th } ; if ($?) {
Original List: 10 -> 20 -> 30 -> 40 -> NULL
Last node deleted successfully.
Updated List after deleting last node: 10 -> 20 -> 30 -> NULL
PS C:\Users\kalpita\OneDrive\Desktop\Kalpita.cpp\F21>

PROGRAM 17 :

Write a C++ program to dynamically allocate memory for an integer array, accept values, and display their sum using pointers.

SOLUTION :

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;

    // Dynamically allocate memory for an integer array
    int* arr = new int[n];

    cout << "Enter " << n << " elements:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> *(arr + i);   // Using pointer arithmetic
    }

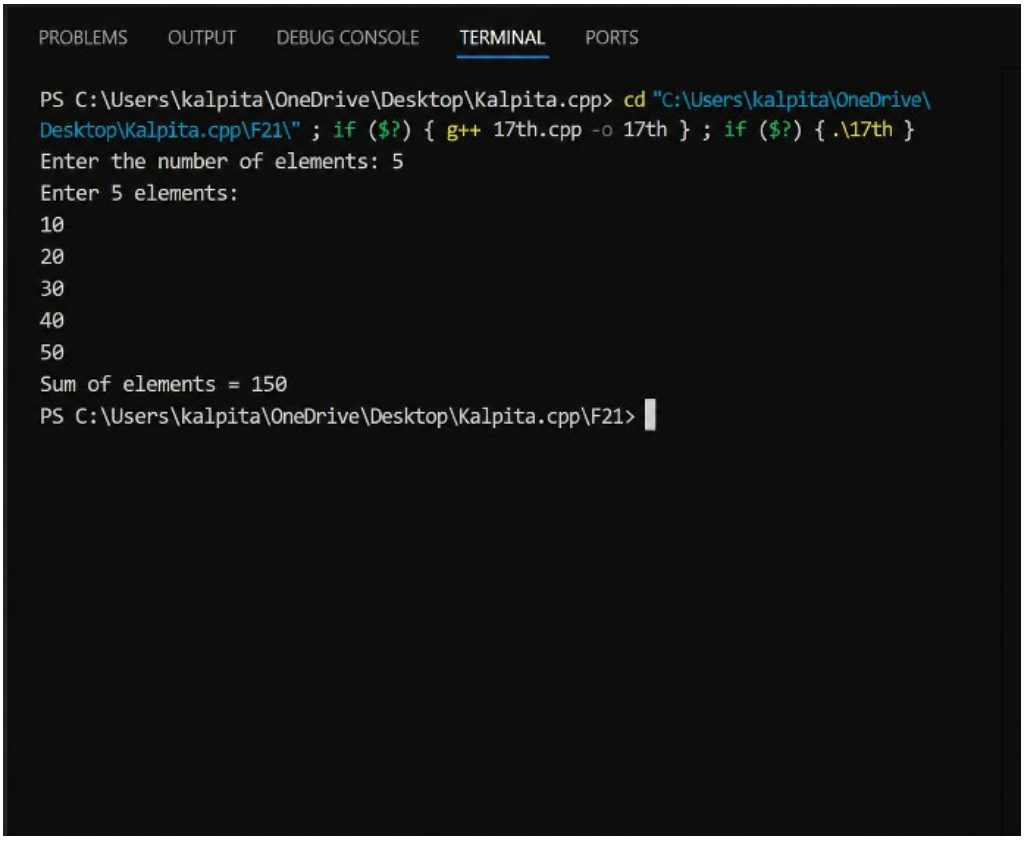
    // Calculate the sum using pointers
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += *(arr + i);   // Accessing array elements using pointers
    }

    cout << "Sum of elements = " << sum << endl;

    // Free the dynamically allocated memory
    delete[] arr;

    return 0;
}
```

OUTPUT :



PROGRAM 18 :

Write a C++ program demonstrating runtime polymorphism by creating a base class Employee and derived classes Manager and Developer, each overriding a function calculateSalary().

SOLUTION :

```
#include <iostream>
using namespace std;

// Base class: Employee
class Employee {
protected:
    string name;
    double baseSalary;

public:
    Employee(const string& n, double salary) : name(n), baseSalary(salary) {
        cout << "Employee created: " << name << endl;
    }

    virtual double calculateSalary() const {
        return baseSalary;
    }

    void displaySalary() const {
        cout << name << "'s salary = " << calculateSalary() << endl;
    }

    virtual ~Employee() {
        cout << "Employee destroyed: " << name << endl;
    }
};

// Derived class: Manager
class Manager : public Employee {
    double bonus;

public:
    Manager(const string& n, double salary, double b) : Employee(n, salary), bonus(b) {
        cout << "Manager created: " << name << endl;
    }

    double calculateSalary() const override {
        return baseSalary + bonus;
    }

    ~Manager() {
        cout << "Manager destroyed: " << name << endl;
    }
};

// Derived class: Developer
class Developer : public Employee {
    double allowance;

public:
    Developer(const string& n, double salary, double a) : Employee(n, salary), allowance(a) {
        cout << "Developer created: " << name << endl;
    }

    double calculateSalary() const override {
        return baseSalary + allowance;
    }

    ~Developer() {
        cout << "Developer destroyed: " << name << endl;
    }
};

int main() {
    Employee* emp;
    Manager mgr("Alice", 50000, 10000);
```

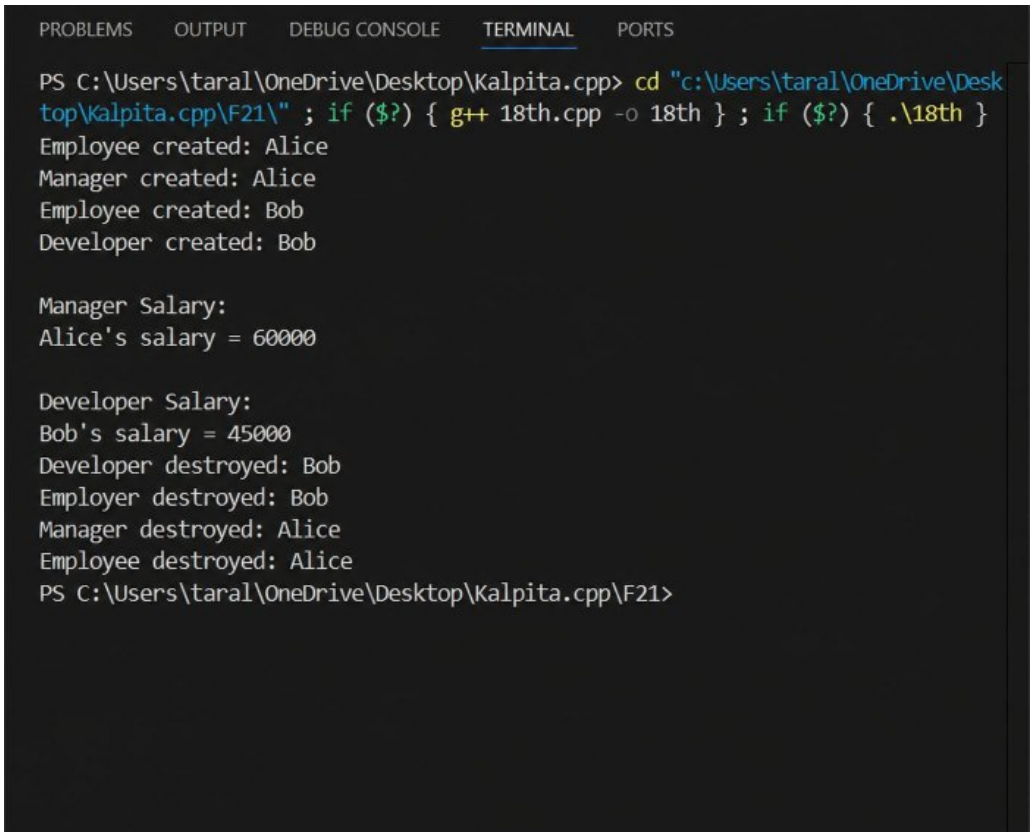
```
    Developer dev("Bob", 40000, 5000);

    cout << "\nManager Salary:\n";
    emp = &mgr;
    emp->displaySalary();

    cout << "\nDeveloper Salary:\n";
    emp = &dev;
    emp->displaySalary();

    return 0;
}
```

OUTPUT :



PROGRAM 19 :

Write a C++ program with a class Car that uses a constructor to initialize its model and brand, and a destructor to display a message when an object is deleted.

SOLUTION :

```
#include <iostream>
using namespace std;

class Car {
private:
    string brand;
    string model;

public:
    // Constructor to initialize brand and model
    Car(string b, string m) {
        brand = b;
        model = m;
        cout << "Car created: " << brand << " " << model << endl;
    }

    // Destructor to display a message when object is deleted
    ~Car() {
        cout << "Car destroyed: " << brand << " " << model << endl;
    }

    // Function to display car details
    void displayInfo() {
        cout << "Brand: " << brand << ", Model: " << model << endl;
    }
};

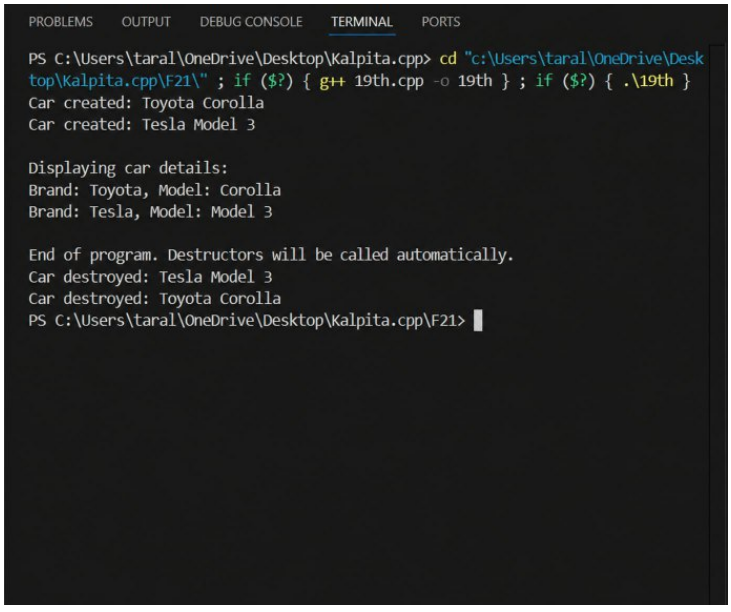
int main() {
    // Creating car objects
    Car car1("Toyota", "Corolla");
    Car car2("Tesla", "Model 3");

    cout << "\nDisplaying car details:" << endl;
    car1.displayInfo();
    car2.displayInfo();

    cout << "\nEnd of program. Destructors will be called automatically." << endl;

    return 0;
}
```

OUTPUT :



PROGRAM 20 :

Write a C++ program to create a binary file to store student records (Roll No., Name, Marks). Include options to update, delete, and view records using fstream.

SOLUTION :

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

class Student {
    int rollNo;
    char name[50];
    float marks;

public:
    void getData() {
        cout << "Enter Roll Number: ";
        cin >> rollNo;
        cin.ignore();
        cout << "Enter Name: ";
        cin.getline(name, 50);
        cout << "Enter Marks: ";
        cin >> marks;
    }

    void showData() const {
        cout << "Roll No: " << rollNo
             << ", Name: " << name
             << ", Marks: " << marks << endl;
    }

    int getRollNo() const {
        return rollNo;
    }
};

// Function to add a new record
void addRecord() {
    ofstream fout("students.dat", ios::binary | ios::app);
    Student s;
    s.getData();
    fout.write(reinterpret_cast<char*>(&s), sizeof(s));
    fout.close();
    cout << "Record added successfully.\n";
}

// Function to display all records
void viewRecords() {
    ifstream fin("students.dat", ios::binary);
    if (!fin) {
        cout << "No records found.\n";
        return;
    }

    Student s;
    cout << "\n---- Student Records ----\n";
    while (fin.read(reinterpret_cast<char*>(&s), sizeof(s))) {
        s.showData();
    }
    fin.close();
}

// Function to update a record
void updateRecord() {
    fstream file("students.dat", ios::binary | ios::in | ios::out);
    if (!file) {
        cout << "File not found.\n";
        return;
    }
```

```
    int roll;
    cout << "Enter Roll Number to update: ";
    cin >> roll;

    Student s;
    bool found = false;

    while (file.read(reinterpret_cast<char*>(&s), sizeof(s))) {
        if (s.getRollNo() == roll) {
            cout << "Current record:\n";
            s.showData();

            cout << "\nEnter new details:\n";
            s.getData();

            // Move file pointer one record back
            int pos = -1 * static_cast<int>(sizeof(s));
            file.seekp(pos, ios::cur);
            file.write(reinterpret_cast<char*>(&s), sizeof(s));

            cout << "Record updated successfully.\n";
            found = true;
            break;
        }
    }

    if (!found)
        cout << "Record not found.\n";

    file.close();
}

// Function to delete a record
void deleteRecord() {
    ifstream fin("students.dat", ios::binary);
    if (!fin) {
        cout << "File not found.\n";
        return;
    }

    ofstream fout("temp.dat", ios::binary);

    int roll;
    cout << "Enter Roll Number to delete: ";
    cin >> roll;

    Student s;
    bool found = false;

    while (fin.read(reinterpret_cast<char*>(&s), sizeof(s))) {
        if (s.getRollNo() != roll) {
            fout.write(reinterpret_cast<char*>(&s), sizeof(s));
        } else {
            found = true;
        }
    }

    fin.close();
    fout.close();

    remove("students.dat");
    rename("temp.dat", "students.dat");

    if (found)
        cout << "Record deleted successfully.\n";
    else
        cout << "Record not found.\n";
}

// Main menu
int main() {
    int choice;

    do {
```

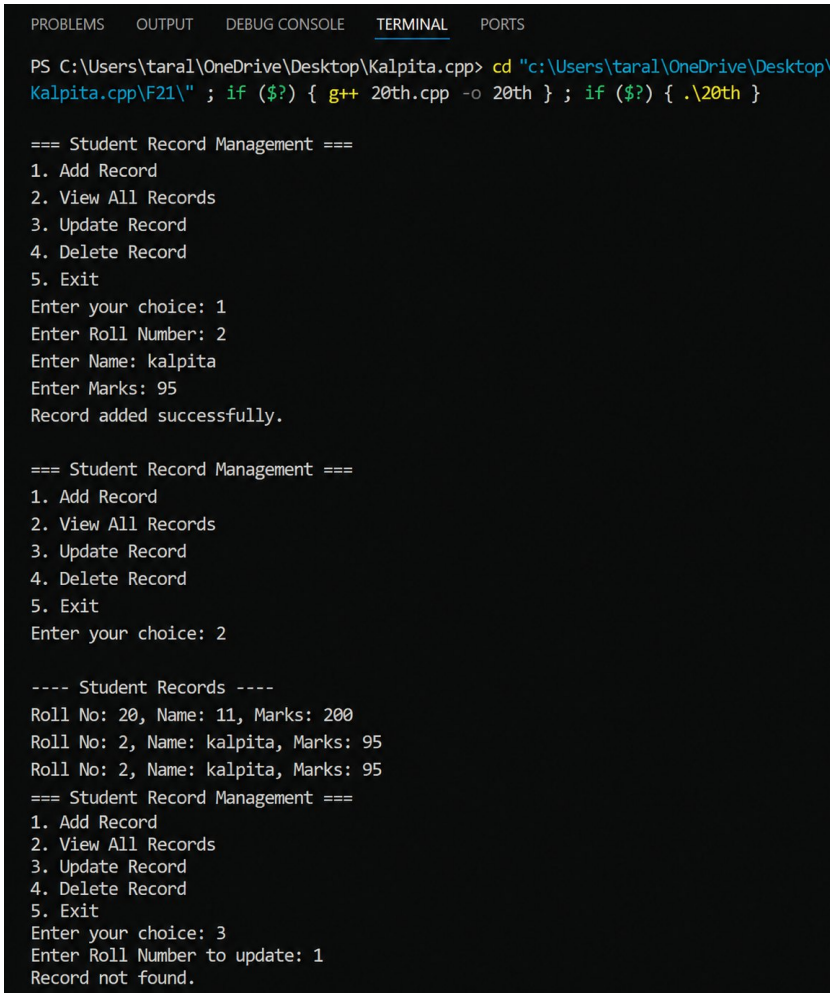
```
        cout << "\n=== Student Record Management ===\n";
        cout << "1. Add Record\n";
        cout << "2. View All Records\n";
        cout << "3. Update Record\n";
        cout << "4. Delete Record\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

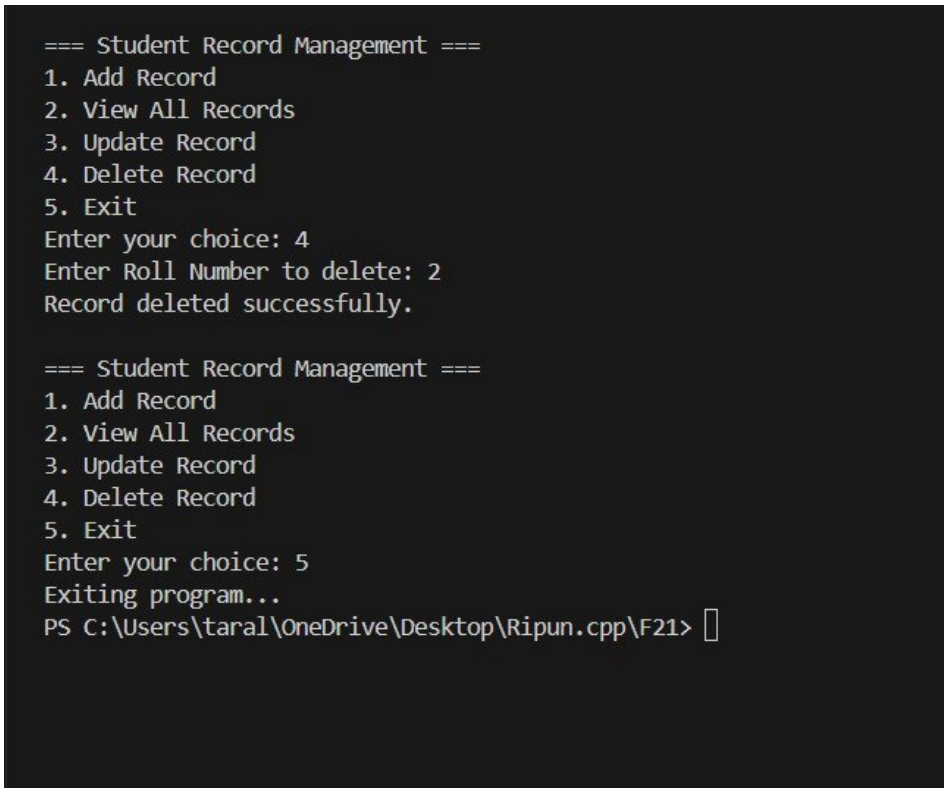
        switch (choice) {
            case 1:
                addRecord();
                break;
            case 2:
                viewRecords();
                break;
            case 3:
                updateRecord();
                break;
            case 4:
                deleteRecord();
                break;
            case 5:
                cout << "Exiting program...\n";
                break;
            default:
                cout << "Invalid choice! Please try again.\n";
        }

    } while (choice != 5);

    return 0;
}
```

OUTPUT :




=== Student Record Management ===
1. Add Record
2. View All Records
3. Update Record
4. Delete Record
5. Exit
Enter your choice: 4
Enter Roll Number to delete: 2
Record deleted successfully.
=== Student Record Management ===
1. Add Record
2. View All Records
3. Update Record
4. Delete Record
5. Exit
Enter your choice: 5
Exiting program...
PS C:\Users\taral\OneDrive\Desktop\Ripun.cpp\F21>

PROGRAM 21 :

Write a menu-driven C++ program to implement queue operations (enqueue, dequeue, and display) using arrays and linked lists.

SOLUTION :

```
#include <iostream>
using namespace std;

#define MAX 5   // Maximum size for the array-based queue

// ---------------------- ARRAY-BASED QUEUE ----------------------
class ArrayQueue {
    int arr[MAX];
    int front, rear;

public:
    ArrayQueue() {
        front = -1;
        rear = -1;
    }

    bool isFull() {
        return rear == MAX - 1;
    }

    bool isEmpty() {
        return front == -1 || front > rear;
    }

    void enqueue(int value) {
        if (isFull()) {
            cout << "Queue Overflow!" << endl;
            return;
        }
        if (front == -1) front = 0;
        arr[++rear] = value;
        cout << "Enqueued (Array): " << value << endl;
    }

    void dequeue() {
        if (isEmpty()) {
            cout << "Queue Underflow!" << endl;
            return;
        }
        cout << "Dequeued (Array): " << arr[front++] << endl;
    }

    void display() {
        if (isEmpty()) {
            cout << "Queue is empty." << endl;
            return;
        }
        cout << "Queue (Array): ";
        for (int i = front; i <= rear; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
};

// ---------------------- LINKED LIST-BASED QUEUE ----------------------
struct Node {
    int data;
    Node* next;
};

class LinkedQueue {
    Node* front;
    Node* rear;

public:
```

```
    LinkedQueue() {
        front = rear = nullptr;
    }

    bool isEmpty() {
        return front == nullptr;
    }

    void enqueue(int value) {
        Node* newNode = new Node;
        newNode->data = value;
        newNode->next = nullptr;

        if (rear == nullptr) {
            front = rear = newNode;
        } else {
            rear->next = newNode;
            rear = newNode;
        }
        cout << "Enqueued (Linked List): " << value << endl;
    }

    void dequeue() {
        if (isEmpty()) {
            cout << "Queue Underflow!" << endl;
            return;
        }
        Node* temp = front;
        cout << "Dequeued (Linked List): " << temp->data << endl;
        front = front->next;
        if (front == nullptr)
            rear = nullptr;
        delete temp;
    }

    void display() {
        if (isEmpty()) {
            cout << "Queue is empty." << endl;
            return;
        }
        cout << "Queue (Linked List): ";
        Node* temp = front;
        while (temp != nullptr) {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }

    ~LinkedQueue() {
        while (!isEmpty()) dequeue();
    }
};

// ---------------------- MAIN MENU ----------------------
int main() {
    int mainChoice, subChoice, value;
    ArrayQueue arrayQueue;
    LinkedQueue linkedQueue;

    do {
        cout << "\n=== Queue Implementation Menu ===" << endl;
        cout << "1. Array-based Queue" << endl;
        cout << "2. Linked List-based Queue" << endl;
        cout << "3. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> mainChoice;

        if (mainChoice == 1) {
            do {
                cout << "\n--- Array Queue Menu ---" << endl;
                cout << "1. Enqueue" << endl;
                cout << "2. Dequeue" << endl;
                cout << "3. Display" << endl;
```

```
                cout << "4. Back to Main Menu" << endl;
                cout << "Enter your choice: ";
                cin >> subChoice;

                switch (subChoice) {
                    case 1:
                        cout << "Enter value to enqueue: ";
                        cin >> value;
                        arrayQueue.enqueue(value);
                        break;
                    case 2:
                        arrayQueue.dequeue();
                        break;
                    case 3:
                        arrayQueue.display();
                        break;
                    case 4:
                        break;
                    default:
                        cout << "Invalid choice!" << endl;
                }
            } while (subChoice != 4);
        }
        else if (mainChoice == 2) {
            do {
                cout << "\n--- Linked List Queue Menu ---" << endl;
                cout << "1. Enqueue" << endl;
                cout << "2. Dequeue" << endl;
                cout << "3. Display" << endl;
                cout << "4. Back to Main Menu" << endl;
                cout << "Enter your choice: ";
                cin >> subChoice;

                switch (subChoice) {
                    case 1:
                        cout << "Enter value to enqueue: ";
                        cin >> value;
                        linkedQueue.enqueue(value);
                        break;
                    case 2:
                        linkedQueue.dequeue();
                        break;
                    case 3:
                        linkedQueue.display();
                        break;
                    case 4:
                        break;
                    default:
                        cout << "Invalid choice!" << endl;
                }
            } while (subChoice != 4);
        }
        else if (mainChoice == 3) {
            cout << "Exiting program..." << endl;
        }
        else {
            cout << "Invalid choice! Please try again." << endl;
        }

    } while (mainChoice != 3);

    return 0;
}
```

OUTPUT :




```
=== Queue Implementation Menu ===
1. Array-based Queue
2. Linked List-based Queue
3. Exit
Enter your choice: 2

--- Linked List Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Back to Main Menu
Enter your choice: 4

=== Queue Implementation Menu ===
1. Array-based Queue
2. Linked List-based Queue
3. Exit
Enter your choice: 3
Exiting program...
PS C:\Users\taral\OneDrive\Desktop\Ripun.cpp\F21>
```

PROGRAM 22 :

Write a menu-driven C++ program to implement stack operations (push, pop, peek, and display) using a linear stack with arrays, a circular stack with arrays, and a stack with a linked list.

SOLUTION :

```
#include <iostream>
using namespace std;

// ---------------- LINEAR STACK (ARRAY) ----------------
class LinearStack {
    int top, size;
    int arr[50];

public:
    LinearStack(int s = 5) { top = -1; size = s; }

    void push(int x) {
        if (top == size - 1) cout << "Linear Stack Overflow\n";
        else arr[++top] = x;
    }

    void pop() {
        if (top == -1) cout << "Linear Stack Underflow\n";
        else cout << "Popped: " << arr[top--] << endl;
    }

    void peek() {
        if (top == -1) cout << "Empty\n";
        else cout << "Top: " << arr[top] << endl;
    }

    void display() {
        if (top == -1) cout << "Empty\n";
        else {
            for (int i = top; i >= 0; i--) cout << arr[i] << " ";
            cout << endl;
        }
    }
};

// ---------------- CIRCULAR STACK (ARRAY) ----------------
class CircularStack {
    int front, rear, size;
    int arr[50];

public:
    CircularStack(int s = 5) {
        size = s; front = rear = -1;
    }

    void push(int x) {
        if ((rear + 1) % size == front) cout << "Circular Stack Overflow\n";
        else {
            if (front == -1) front = 0;
            rear = (rear + 1) % size;
            arr[rear] = x;
        }
    }

    void pop() {
        if (front == -1) cout << "Circular Stack Underflow\n";
        else {
            cout << "Popped: " << arr[rear] << endl;
            if (front == rear) front = rear = -1;
            else rear = (rear - 1 + size) % size;
        }
    }

    void peek() {
        if (front == -1) cout << "Empty\n";
        else cout << "Top: " << arr[rear] << endl;
```

```
    }

    void display() {
        if (front == -1) cout << "Empty\n";
        else {
            int i = rear;
            while (true) {
                cout << arr[i] << " ";
                if (i == front) break;
                i = (i - 1 + size) % size;
            }
            cout << endl;
        }
    }
};

// ---------------- LINKED LIST STACK ----------------
class Node {
public:
    int data;
    Node *next;
};

class LinkedListStack {
    Node *top;

public:
    LinkedListStack() { top = NULL; }

    void push(int x) {
        Node *n = new Node();
        n->data = x;
        n->next = top;
        top = n;
    }

    void pop() {
        if (top == NULL) cout << " Underflow\n";
        else {
            cout << "Popped: " << top->data << endl;
            Node *t = top;
            top = top->next;
            delete t;
        }
    }

    void peek() {
        if (top == NULL) cout << "Empty\n";
        else cout << "Top: " << top->data << endl;
    }

    void display() {
        Node *p = top;
        if (!p) cout << "Empty\n";
        else {
            while (p) {
                cout << p->data << " ";
                p = p->next;
            }
            cout << endl;
        }
    }
};

// ---------------- MAIN MENU ----------------
int main() {
    LinearStack L;
    CircularStack C;
    LinkedListStack K;
    int choice, stype, val;

    while (true) {
        cout << "\n1. Linear Stack\n2. Circular Stack\n3. Linked List Stack\n4. Exit\nChoice: ";
        cin >> stype;
```

```
        if (stype == 4) break;

        cout << " 1.Push 2.Pop 3.Peek 4.Display : ";
        cin >> choice;

        switch (stype) {
            case 1:
                if (choice == 1) { cout << "Enter value: "; cin >> val; L.push(val); }
                else if (choice == 2) L.pop();
                else if (choice == 3) L.peek();
                else L.display();
                break;
            case 2:
                if (choice == 1) { cout << "Enter value: "; cin >> val; C.push(val); }
                else if (choice == 2) C.pop();
                else if (choice == 3) C.peek();
                else C.display();
                break;
            case 3:
                if (choice == 1) { cout << "Enter value: "; cin >> val; K.push(val); }
                else if (choice == 2) K.pop();
                else if (choice == 3) K.peek();
                else K.display();
                break;
            default: cout << " Invalid\n";
        }
    }

    return 0;
}
```

OUTPUT :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\user\Desktop\practical test> ./22TH.EXE

 1. Linear Stack
 2. Circular Stack
 3. Linked List Stack
 4. Exit
 Choice: 1
 1.Push  2.Pop  3.Peek  4.Display : 2
 Linear Stack Underflow

 1. Linear Stack
 2. Circular Stack
 3. Linked List Stack
 4. Exit
PS C:\Users\user\Desktop\practical test> ./22TH.EXE

 1. Linear Stack
 2. Circular Stack
 3. Linked List Stack
 4. Exit
 Choice: 1
 1.Push  2.Pop  3.Peek  4.Display : 2
 Linear Stack Underflow

 1. Linear Stack
 2. Circular Stack
 3. Linked List Stack
 4. Exit
 Choice: 2
 1.Push  2.Pop  3.Peek  4.Display : 1
 Enter value: 2

 1. Linear Stack
 2. Circular Stack
 3. Linked List Stack
 4. Exit
PS C:\Users\user\Desktop\practical test> ./22TH.EXE

 1. Linear Stack
 2. Circular Stack
 3. Linked List Stack
 4. Exit
 Choice: 1
 1.Push  2.Pop  3.Peek  4.Display : 2
 Linear Stack Underflow

 1. Linear Stack
 2. Circular Stack
 3. Linked List Stack
 4. Exit
 Choice: 2
 1.Push  2.Pop  3.Peek  4.Display : 1
 Enter value: 2
```

```
1. Linear Stack
2. Circular Stack
3. Linked List Stack
4. Exit
Choice: 2
1.Push  2.Pop  3.Peek  4.Display : 1
Enter value: 2

1. Linear Stack
2. Circular Stack
3. Linked List Stack
3. Linked List Stack
4. Exit
Choice: 1
1.Push  2.Pop  3.Peek  4.Display : 2
Linear Stack Underflow

1. Linear Stack
2. Circular Stack
3. Linked List Stack
4. Exit
Choice: 2
1.Push  2.Pop  3.Peek  4.Display : 1
Enter value: 2

1. Linear Stack
2. Circular Stack
3. Linked List Stack
4. Exit
Choice: 2
1.Push  2.Pop  3.Peek  4.Display : 1
Enter value: 2

1. Linear Stack
2. Circular Stack
Choice: 2
1.Push  2.Pop  3.Peek  4.Display : 1
Enter value: 2

1. Linear Stack
2. Circular Stack
1. Linear Stack
2. Circular Stack
3. Linked List Stack
3. Linked List Stack
4. Exit
4. Exit
Choice: 4
PS C:\Users\user\Desktop\practical test>
```

Assignment: SQL Commands with Usage and Output

1.Creating a Table

Usage:
This command is used to create a table named Employees with columns ID, Name, Role, Salary, Joining_Date, and Company. Each column is defined with its data type and constraints.

Command with output:

```
sql


mysql> CREATE TABLE Employees (
    -> ID INT PRIMARY KEY,
    -> Name VARCHAR(50),
    -> Role VARCHAR(50),
    -> Salary INT,
    -> Joining_Date DATE,
    -> Company VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.01 sec)
```

2. Insert Records

Usage:
Inserts a new record into the Employees table with specific values for each column.

Command with output:

```
sql

mysql> INSERT INTO Employees (ID, Name, Role, Salary, Joining_Date, Company)
    -> VALUES
    -> (1, 'Rahul Sharma', 'Developer', 50000, '2024-11-01', 'Skillarger'),
    -> (2, 'Anjali Mehta', 'Designer', 45000, '2024-10-15', 'Skillarger'),
    -> (3, 'Amitabh Sinha', 'Manager', 70000, '2024-09-20', 'Skillarger'),
    -> (4, 'Priya Singh', 'HR', 40000, '2024-07-10', 'Skillarger'),
    -> (5, 'Ravi Patel', 'Analyst', 55000, '2024-08-05', 'Skillarger');
Query OK, 5 rows affected (0.01 sec)
```

3. Select All Records

This command retrieves all rows and columns from the Employees table, displaying the complete dataset.s all the records and columns from the Employees table.

Command with output:

```
mysql> SELECT * FROM Employees;
+----+---------------+----------+--------+--------------+-------------+
| ID | Name          | Role     | Salary | Joining_Date | Company     |
+----+---------------+----------+--------+--------------+-------------+
|  1 | Rahul Sharma  | Developer |  50000 | 2024-11-01   | Skillarger  |
|  2 | Anjali Mehta  | Designer  |  45000 | 2024-10-15   | Skillarger  |
|  3 | Amitabh Sinha | Manager   |  70000 | 2024-09-20   | Skillarger  |
|  4 | Priya Singh   | HR        |  40000 | 2024-07-10   | Skillarger  |
|  5 | Ravi Patel    | Analyst   |  55000 | 2024-08-05   | Skillarger  |
+----+---------------+----------+--------+--------------+-------------+
5 rows in set (0.01 sec)
```

4. Select Specific Columns

Usage:
This command retrieves only the `Name` and `Role` columns for all employees, ignoring the rest of the data.

Command with output:

```
sql

mysql> SELECT Name, Role FROM Employees;
+---------------+-----------+
| Name          | Role      |
+---------------+-----------+
| Rahul Sharma  | Developer |
| Anjali Mehta  | Designer  |
| Amitabh Sinha | Manager   |
| Priya Singh   | HR        |
| Ravi Patel    | Analyst   |
+---------------+-----------+
5 rows in set (0.01 sec)
```

5.Filter by Salary

Usage:

This command retrieves records of employees whose salaries are greater than 50,000.

Command with output:

```
sql


mysql> SELECT * FROM Employees WHERE Salary > 50000;
+----+----------------+---------+--------+--------------+-------------+
| ID | Name           | Role    | Salary | Joining_Date | Company     |
+----+----------------+---------+--------+--------------+-------------+
|  3 | Amitabh Sinha  | Manager |  70000 | 2024-09-20   | Skillarger  |
|  5 | Ravi Patel     | Analyst |  55000 | 2024-08-05   | Skillarger  |
+----+----------------+---------+--------+--------------+-------------+
2 rows in set (0.01 sec)
```

6.Update a Record

Usage:

This command updates the salary of the employee with ID = 2 to 60,000.

Command with output:

```
sql


mysql> UPDATE Employees SET Salary = 60000 WHERE ID = 2;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

7. Delete a Record

Usage:

This command deletes the record of the employee with ID = 4.

Command with output:

```
sql


mysql> DELETE FROM Employees WHERE ID = 4;
Query OK, 1 row affected (0.00 sec)
```

8.Count Total Records

Usage:
This command counts the total number of rows (records) in the Employees table.
Command with output:

```
sql


mysql> SELECT COUNT(*) FROM Employees;
+----------+
| COUNT(*) |
+----------+
|        4 |
+----------+
1 row in set (0.00 sec)
```

9. Sort Records

Usage:
This command sorts the rows of the Employees table in descending order based on the Salary column.
Command with output:

```
sql


mysql> SELECT * FROM Employees ORDER BY Salary DESC;
+----+---------------+----------+--------+--------------+------------+
| ID | Name          | Role     | Salary | Joining_Date | Company    |
+----+---------------+----------+--------+--------------+------------+
|  3 | Amitabh Sinha | Manager  |  70000 | 2024-09-20   | Skillarger |
|  2 | Anjali Mehta  | Designer |  60000 | 2024-10-15   | Skillarger |
|  5 | Ravi Patel    | Analyst  |  55000 | 2024-08-05   | Skillarger |
|  1 | Rahul Sharma  | Developer|  50000 | 2024-11-01   | Skillarger |
+----+---------------+----------+--------+--------------+------------+
4 rows in set (0.01 sec)
```

10. Find Maximum Salary

Usage:
This command retrieves the maximum salary value from the Employees table.
Command with output:

```
sql


mysql> SELECT MAX(Salary) AS Highest_Salary FROM Employees;
+----------------+
| Highest_Salary |
+----------------+
|          70000 |
+----------------+
1 row in set (0.00 sec)
```

11. Group by Role

Usage:
This command groups the data by the Role column and counts the number of employees in each role.
Command with output:

```
sql


mysql> SELECT Role, COUNT(*) AS Total FROM Employees GROUP BY Role;
+-----------+-------+
| Role      | Total |
+-----------+-------+
| Analyst   |     1 |
| Designer  |     1 |
| Developer |     1 |
| Manager   |     1 |
+-----------+-------+
4 rows in set (0.01 sec)
```

12. Drop the Table

Usage:
This command deletes the Employees table and all its data permanently from the database.
Command with output:

```
sql


mysql> DROP TABLE Employees;
Query OK, 0 rows affected (0.01 sec)
```

13. Add a New Column

Usage:
This command adds a new column Email to the Employees table.
Command with output:

```
sql


mysql> ALTER TABLE Employees ADD Email VARCHAR(100);
Query OK, 0 rows affected (0.02 sec)
```

14. Update Email Addresses

Usage:
This command generates email addresses for employees by combining their names and the company domain.

Command with output:

```
sql


mysql> UPDATE Employees
    -> SET Email = CONCAT(LOWER(Name), '@skillarger.com');
Query OK, 4 rows affected (0.01 sec)
Rows matched: 4  Changed: 4  Warnings: 0
```

15. Select Records with Email Column

Usage:
This command retrieves the ID, Name, and Email columns from the Employees table.
Command with output:

```
sql


mysql> SELECT ID, Name, Email FROM Employees;
+----+----------------+----------------------------+
| ID | Name           | Email                      |
+----+----------------+----------------------------+
|  1 | Rahul Sharma   | rahul sharma@skillarger.com |
|  2 | Anjali Mehta   | anjali mehta@skillarger.com |
|  3 | Amitabh Sinha  | amitabh sinha@skillarger.com|
|  5 | Ravi Patel     | ravi patel@skillarger.com   |
+----+----------------+----------------------------+
4 rows in set (0.00 sec)
```

16. Filter by Joining Date

Usage:
This command retrieves records of employees who joined after September 1, 2024.
Command with output:

```
sql


mysql> SELECT * FROM Employees WHERE Joining_Date > '2024-09-01';
+----+----------------+-----------+--------+--------------+-------------+
| ID | Name           | Role      | Salary | Joining_Date | Company     |
+----+----------------+-----------+--------+--------------+-------------+
|  1 | Rahul Sharma   | Developer |  50000 | 2024-11-01   | Skillarger  |
|  2 | Anjali Mehta   | Designer  |  60000 | 2024-10-15   | Skillarger  |
+----+----------------+-----------+--------+--------------+-------------+
2 rows in set (0.01 sec)
```

17. Rename Column

Usage:
This command renames the column Email to Contact_Email in the Employees table.
Command with output:

```
sql


mysql> ALTER TABLE Employees RENAME COLUMN Email TO Contact_Email;
Query OK, 0 rows affected (0.02 sec)
```

18. Add a New Employee

Usage:
This command inserts a new record for Neha Kapoor with her details into the Employees table.
Command with output:

```
sql


mysql> INSERT INTO Employees (ID, Name, Role, Salary, Joining_Date, Company)
    -> VALUES (6, 'Neha Kapoor', 'Tester', 48000, '2024-11-20', 'Skillarger');
Query OK, 1 row affected (0.01 sec)
```

19. Show New Table with Updated Records

Usage:
This command displays all records from the updated Employees table, including the new data.
Command with output:

```
Sql

mysql> SELECT * FROM Employees;
+----+---------------+----------+--------+--------------+------------+
| ID | Name          | Role     | Salary | Joining_Date | Company    |
+----+---------------+----------+--------+--------------+------------+
|  1 | Rahul Sharma  | Developer |  50000 | 2024-11-01   | Skillarger |
|  2 | Anjali Mehta  | Designer  |  60000 | 2024-10-15   | Skillarger |
|  3 | Amitabh Sinha | Manager   |  70000 | 2024-09-20   | Skillarger |
|  5 | Ravi Patel    | Analyst   |  55000 | 2024-08-05   | Skillarger |
|  6 | Neha Kapoor   | Tester    |  48000 | 2024-11-20   | Skillarger |
+----+---------------+----------+--------+--------------+------------+
5 rows in set (0.01 sec)
```

20. Create a View

Usage:

This command creates a view HighEarners to show employees earning more than 50,000.

Command with output:

```
sql


mysql> CREATE VIEW HighEarners AS
    -> SELECT Name, Role, Salary
    -> FROM Employees
    -> WHERE Salary > 50000;
Query OK, 0 rows affected (0.01 sec)
```

21. Display View Data

Usage:

This command retrieves data from the HighEarners view, displaying only high-salary employees.

Command with output:

```
sql


mysql> SELECT * FROM Employees;
+----+----------------+----------+--------+--------------+-------------+
mysql> SELECT * FROM HighEarners;
+----------------+----------+--------+
| Name           | Role     | Salary |
+----------------+----------+--------+
| Anjali Mehta   | Designer |  60000 |
| Amitabh Sinha  | Manager  |  70000 |
| Ravi Patel     | Analyst  |  55000 |
+----------------+----------+--------+
3 rows in set (0.00 sec)
```

21 SQL commands We use on this project

1. **Create Table** =`CREATE TABLE Employees (...);`
2. **Insert Records** =`INSERT INTO Employees (...) VALUES (...);`
3. **Select All Records** =`SELECT * FROM Employees;`
4. **Select Specific Columns** =`SELECT Name, Role FROM Employees;`
5. **Filter by Salary** =`SELECT * FROM Employees WHERE Salary > 50000;`
6. **Update a Record** =`UPDATE Employees SET Salary = 60000 WHERE ID = 2;`
7. **Delete a Record** =`DELETE FROM Employees WHERE ID = 4;`
8. **Count Total Records** =`SELECT COUNT(*) FROM Employees;`
9. **Sort Records** =`SELECT * FROM Employees ORDER BY Salary DESC;`
10. **Find Maximum Salary** =`SELECT MAX(Salary) AS Highest_Salary FROM Employees;`
11. **Group by Role** =`SELECT Role, COUNT(*) AS Total FROM Employees GROUP BY Role;`
12. **Drop Table** =`DROP TABLE Employees;`
13. **Add a New Column** =`ALTER TABLE Employees ADD Email VARCHAR(100);`
14. **Update Email Addresses** =`UPDATE Employees SET Email = CONCAT(LOWER(Name), '@skillarger.com');`
15. **Select Records with Email Column** =`SELECT ID, Name, Email FROM Employees;`
16. **Filter by Joining Date** =`SELECT * FROM Employees WHERE Joining_Date > '2024-09-01';`
17. **Rename Column** =`ALTER TABLE Employees RENAME COLUMN Email TO Contact_Email;`
18. **Add a New Employee** =`INSERT INTO Employees (...) VALUES (...);`
19. **Show Updated Table** =`SELECT * FROM Employees;`
20. **Create a View** =`CREATE VIEW HighEarners AS SELECT ...;`
21. **Display View Data** =`SELECT * FROM HighEarners;`